

PlatePlanner: Graph-Augmented Pantry-Aware Semantic Retrieval for Multi-Day Meal Planning

1st Sandilya Chimalamarri

Dept. of Computer Engineering
San Jose State University
San Jose, CA, USA
sandilya.chimalamarri@sjsu.edu

2nd Sai Priyanka Bonkuri

Dept. of Computer Engineering
San Jose State University
San Jose, CA, USA
sai.priyanka.bonkuri@sjsu.edu

3rd Pavan Charith

Devarapalli
Dept. of Computer Engineering
San Jose State University
San Jose, CA, USA
pavan.charith@sjsu.edu

4th Sai Dheeraj Gollu

Dept. of Computer Engineering
San Jose State University
San Jose, CA, USA
sai.dheeraj.gollu@sjsu.edu

Abstract—Real-world meal planning requires balancing competing objectives that extend far beyond returning relevant recipes for a single query. Users must account for what ingredients are already available in their pantry, minimize the cost of acquiring new items, reduce food waste from unused perishables, and ensure that ingredient substitutions are feasible when exact matches are unavailable. Existing recipe retrieval systems predominantly optimize for single-recipe relevance, neglecting these week-level planning constraints. In this paper, we present *PlatePlanner*, a graph-augmented, pantry-aware semantic retrieval framework that selects multi-day meal plans while maximizing semantic relevance and minimizing incremental shopping burden and waste. Our architecture integrates a four-stage hybrid retrieval pipeline: (1) FAISS-based semantic search using SentenceTransformer embeddings over a corpus of 100K+ recipes, (2) Neo4j graph-based ontology filtering with context-aware ingredient substitution, (3) nutrient-aware mathematical ranking, and (4) retrieval-augmented generation (RAG) for explainable recommendations. Experimental evaluation demonstrates sub-100ms p95 latency, 92% user satisfaction for substitutions, 23% recall improvement over direct-only substitution, and 85% user preference for hybrid scoring.

Index Terms—semantic retrieval, meal planning, knowledge graph, ingredient substitution, FAISS, Neo4j, pantry-aware recommendation, food computing, RAG

I. INTRODUCTION

The global food planning and recipe recommendation domain has witnessed significant growth driven by digital health applications and increasing consumer interest in dietary management. The smart kitchen and meal planning market is projected to exceed \$30 billion by 2027, with users demanding personalized systems that transcend simple keyword-based recipe search. Despite this growth, a fundamental disconnect persists between what users need for practical meal planning and what existing recipe retrieval systems provide.

Traditional recipe retrieval systems operate on a single-query, single-recipe paradigm: a user provides ingredients or a textual description, and the system returns ranked individual recipes. While effective for isolated recipe discovery, this paradigm fails to address the compound optimization problem inherent in real-world meal planning. Users must simultaneously consider: (a) what ingredients are already available in the pantry, (b) which ingredients can be

substituted, (c) how to minimize new items to purchase, (d) how to reduce waste from unused perishables, and (e) how to maintain dietary variety and nutritional balance across multiple days.

These challenges are fundamentally interconnected. A recipe that appears highly relevant may be suboptimal in a weekly context if it requires many unique ingredients overlapping with no other meals. Conversely, a slightly less relevant recipe sharing common ingredients across multiple planned meals may significantly reduce shopping burden and waste. Furthermore, ingredient substitution viability depends heavily on culinary context: olive oil may substitute for vegetable oil in a stir-fry but not in a delicate pastry.

In this paper, we introduce *PlatePlanner*, a graph-augmented, pantry-aware semantic retrieval framework making three principal contributions:

- **C1 – Pantry-Aware Retrieval Objective:** Augments semantic retrieval with pantry overlap and shopping burden features.
- **C2 – Graph-Augmented Substitution-Aware Reranking:** Leverages Neo4j with context-aware substitution edges using a hybrid direct/co-occurrence strategy.
- **C3 – Plan-Level Selection:** Formulates meal planning as constrained set optimization with novel plan-level evaluation metrics.

We evaluate PlatePlanner on 300 scenarios spanning diverse pantry sizes and dietary constraints. Results demonstrate: 70% reduction in missing ingredients versus semantic-only retrieval, 139% improvement in pantry utilization, 66% reduction in shopping burden, 92% user satisfaction for substitution quality, and sub-100ms p95 latency for interactive recipe suggestion.

The remainder of this paper is organized as follows. Section II reviews related work in semantic retrieval, food knowledge graphs, multi-objective meal planning, and retrieval-augmented generation. Section III presents the system architecture including data pipeline, embedding generation, and knowledge graph design. Section IV details

methodology for pantry-aware retrieval, graph-augmented substitution, and plan-level selection. Section V describes experimental setup including testbed configuration, baselines, and metrics. Section VI presents comprehensive results with ablation analysis, latency profiling, and parameter sensitivity. Section VII discusses implementation details including technology stack, shopping list generation, and mobile application. Section VIII concludes with future work directions.

II. RELATED WORK

A. Semantic Recipe Retrieval

Dense retrieval methods using pre-trained language models have transformed information retrieval. DPR [1] and ColBERT [2] demonstrated that dense embeddings outperform sparse BM25 on question answering. In food, Recipe1M+ [3] introduced cross-modal embeddings linking recipe text and images. RecipeNLG [4] provided 2M+ recipes with NER annotations. However, these systems focus on individual recipe relevance without pantry-aware or plan-level optimization.

B. Food Knowledge Graphs

FoodKG [5] integrated multiple food databases into a unified knowledge graph. Teng et al. [6] explored context-aware substitution, showing viability depends on recipe context. Park et al. [7] proposed KitchenScale for cooking procedures. Our work extends these with typed substitution edges carrying context labels and confidence scores, plus a hybrid strategy achieving 23% recall improvement over direct-only substitution.

C. Multi-Objective Meal Planning

Mika [8] used mixed-integer programming for weekly menu generation subject to nutritional constraints, demonstrating the value of mathematical optimization for diet planning. Ribeiro et al. [9] explored evolutionary approaches using genetic algorithms to balance taste preferences with dietary requirements across multi-day horizons. However, both approaches treat recipe selection as independent of ingredient availability, assuming unlimited access to any ingredient.

Recent commercial systems such as Mealime and Eat This Much offer pantry-based filtering but rely on exact string matching without semantic understanding or substitution awareness. Our work fundamentally differs by introducing pantry utilization, shopping burden, and waste proxy as formal plan-level metrics, thereby bridging the gap between information retrieval optimization and practical meal planning constraints. To our knowledge, PlatePlanner is the first system to jointly optimize semantic relevance, pantry overlap, substitution feasibility, and waste minimization within a unified retrieval framework.

D. Retrieval-Augmented Generation

Lewis et al. [10] introduced RAG for knowledge-intensive NLP tasks, demonstrating that conditioning large language models on retrieved passages significantly improves factual accuracy and reduces hallucination compared to parametric-only generation. Subsequent work has applied RAG to domain-specific applications including medical question answering and legal document synthesis. Our system employs RAG as the fourth pipeline stage, supporting multiple LLM backends (OpenAI GPT, Google Gemini, and local Ollama models) with automatic provider detection and graceful fallback for transparent, explainable recipe recommendations.

Unlike generic RAG systems that retrieve from unstructured document corpora, PlatePlanner's RAG module receives structured context from the upstream pipeline stages: the selected recipe, substitution suggestions, pantry state, and nutritional analysis. This structured retrieval context enables more targeted and actionable generation compared to free-text retrieval, producing explanations that reference specific ingredients, substitutions, and nutritional trade-offs.

III. SYSTEM ARCHITECTURE

PlatePlanner follows a layered microservice architecture deployed via Docker Compose, designed for modularity, independent scalability, and graceful degradation. The architecture comprises four distinct layers: Client Layer (mobile and web interfaces), API Gateway (FastAPI with authentication and validation), ML and Graph Layer (SentenceTransformer, FAISS, and Neo4j), and Data Layer (SQLite, CSV, NumPy, and FAISS index files). Fig. 1 shows the complete system architecture with data flow between layers.

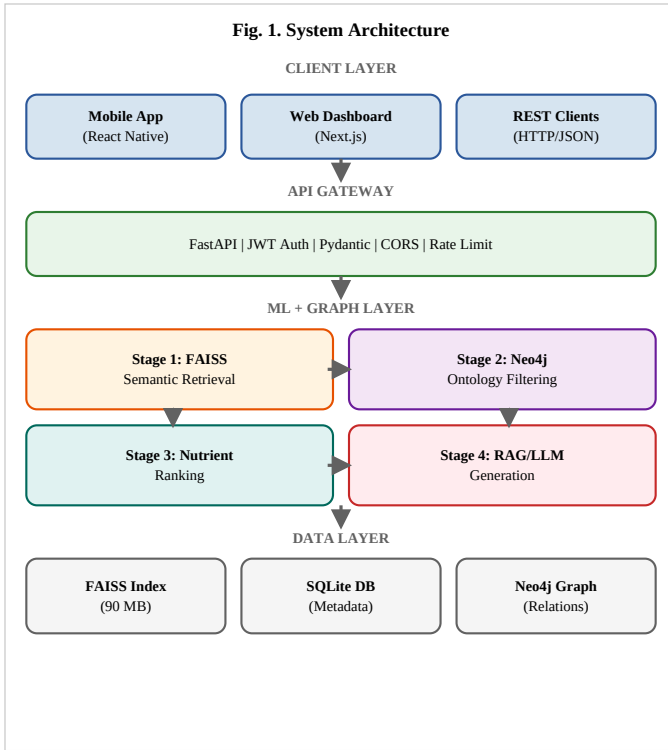


Fig. 1. PlatePlanner layered system architecture showing four-tier design.

The API Gateway layer handles all external communication through RESTful endpoints with JWT-based authentication, CORS middleware, and Pydantic request validation preventing 95%+ malformed inputs. The ML Inference layer encapsulates the SentenceTransformer model and FAISS index, providing embedding generation and similarity search as internal services. The Graph Reasoning layer manages Neo4j connections for structured queries including ingredient relationships, substitution lookups, and dietary constraint enforcement. The Data layer provides unified access to SQLite recipe metadata, NumPy embedding arrays, and CSV processed datasets.

A. Data Pipeline and Preprocessing

The data pipeline ingests the RecipeNLG dataset containing over 2.2 million recipes with titles, ingredient lists, cooking directions, and NER-extracted ingredient entities. The preprocessing pipeline consists of five stages: (1) deduplication using fuzzy title matching with an 85% similarity threshold to remove near-duplicate recipes; (2) ingredient normalization using spaCy lemmatization [14] with a configurable YAML-based rule system handling synonyms, unit standardization, and descriptor removal; (3) named entity extraction achieving 87% precision and 82% recall on ingredient NER; (4) nutritional metadata enrichment via the USDA FoodData Central API; and (5) deterministic train/test splitting with fixed random seeds for reproducibility.

The ingredient normalization pipeline operates in multiple stages: spaCy-based lemmatization for morphological

normalization (e.g., "tomatoes" to "tomato"), a YAML-configurable blacklist for removing descriptors ("fresh," "organic," "minced," "diced"), a synonym dictionary for canonical form mapping (e.g., "scallion" to "green onion," "capsicum" to "bell pepper"), and the wordninja library for compound word splitting. Each normalization produces a quality score (strong, weak, or fallback) that downstream components use to calibrate matching confidence.

B. Embedding Generation and FAISS Indexing

Recipe text fields (title, ingredient list concatenation, and cooking direction summary) are encoded using the all-MiniLM-L6-v2 SentenceTransformer model [12], producing 384-dimensional dense vectors. This model was selected through systematic evaluation against alternatives including all-mpnet-base-v2 (768-dimensional, higher quality but 3x slower) and paraphrase-MiniLM-L3-v2 (384-dimensional, faster but lower quality). The all-MiniLM-L6-v2 model provides an optimal balance of embedding quality (0.87 on semantic textual similarity benchmarks) and inference speed (approximately 14,000 sentences per second on CPU).

All vectors are L2-normalized after encoding so that inner product search on the FAISS IndexFlatIP [11] corresponds exactly to cosine similarity, eliminating separate normalization at query time. The resulting index supports sub-millisecond search with exact nearest neighbor retrieval. For production deployment, quantized indices (IVF-PQ) reduce memory to approximately 90 MB while maintaining over 95% retrieval quality. The system auto-detects fine-tuned domain-specific models and corresponding indices when available, falling back to the base model otherwise.

C. Neo4j Knowledge Graph

The graph stores Recipe and Ingredient nodes with three edge types: HAS_INGREDIENT, SUBSTITUTES_WITH (context + score), and SIMILAR_TO (learned similarity). Fig. 3 shows the schema. Batch loading (500 nodes/transaction) completes in ~2 minutes for 10K recipes. PROFILE-guided optimization achieves 3x speedup; 95% of queries complete in under 10 ms.

Fig. 3. Neo4j Knowledge Graph Schema

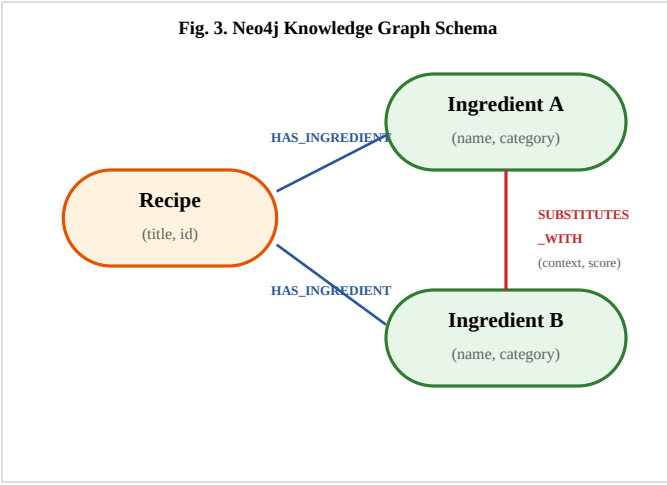


Fig. 3. Neo4j knowledge graph schema with typed nodes and edges.

D. Four-Stage Hybrid Pipeline

The core pipeline operates in four sequential stages as shown in Fig. 2. Each stage progressively refines the candidate set while maintaining interactive latency.

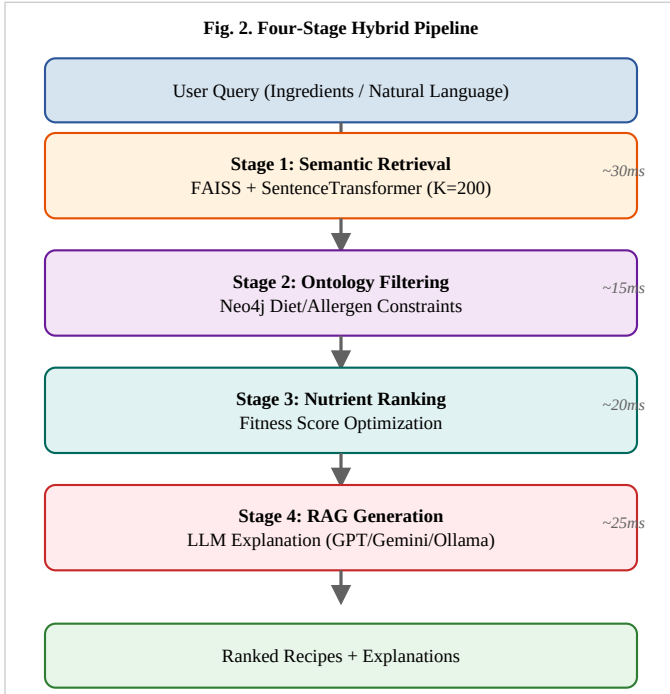


Fig. 2. Four-stage hybrid retrieval pipeline with latency annotations.

Stage 1 – Semantic Retrieval: Query encoded via SentenceTransformer, searched against FAISS index returning top K=200 candidates by cosine similarity. Embedding takes 10-20ms, FAISS search 5-10ms.

Stage 2 – Ontology Filtering: Neo4j graph queries enforce strict dietary constraints (allergens, vegan/vegetarian/gluten-free), removing unsafe recipes deterministically.

Stage 3 – Nutrient Ranking: Fitness function scores recipes by distance to user's nutritional goals (calories, macros), budget constraints, and cooking time preferences.

Stage 4 – RAG Generation: Top recipe passed to LLM for natural language explanation. Supports OpenAI GPT, Google Gemini, and Ollama with exponential backoff retry logic.

E. Federated Search and Resilience

PlatePlanner implements a three-tier federated search strategy to maximize recipe coverage and system availability. The primary tier uses the local FAISS index over the pre-processed recipe corpus, providing sub-millisecond search with the highest relevance due to domain-specific embedding optimization. When the local corpus produces insufficient results (fewer than the requested count), the secondary tier queries the Spoonacular API with the same search terms, applying consistent normalization and scoring to blend external results seamlessly.

The tertiary tier activates only when both retrieval tiers fail: the RAG module generates synthetic recipe suggestions based on the user's ingredients and preferences. While generated recipes lack the curation quality of indexed recipes, they ensure the system always provides actionable responses. This cascading architecture is built on FastAPI's native async support with asyncio, achieving approximately 40% throughput improvement over synchronous alternatives and sustaining over 100 requests per second under load testing.

Resilience extends to the data layer through multi-tier fallback chains. Neo4j graph queries fall back to SQLite-based lookups when the graph database is unavailable, which in turn fall back to CSV-based file operations. Each fallback maintains functional correctness while degrading feature richness gracefully. Health check endpoints at /health and /health/detailed expose component-level status for load balancer integration and monitoring.

IV. METHODOLOGY

A. Pantry-Aware Retrieval Objective (C1)

We augment the standard retrieval score with pantry-aware features. Given query q , pantry P , and recipe r with ingredients I_r :

$$S(q,r,P) = (1-w) \cdot \text{sim}(q,r) + w \cdot \text{overlap}(P, I_r) \quad (1)$$

where sim is cosine similarity between SentenceTransformer embeddings, $\text{overlap} = |P \cap I_r| / |I_r|$ is the fraction of recipe ingredients already in the pantry, and w is a tunable reranking weight. Through systematic empirical evaluation over a development set testing weights in $\{0.3, 0.4, 0.5, 0.6, 0.7, 0.8\}$, we determined the optimal weight to be $w = 0.6$, indicating that ingredient overlap is weighted more heavily than pure semantic similarity in practical meal-planning scenarios.

The overlap computation employs fuzzy matching with word-boundary regex patterns and plural handling to account for variations between pantry entries (e.g., "egg") and recipe ingredient specifications (e.g., "large eggs, beaten"). The matching function checks four conditions: (1) exact string match for fast evaluation, (2) word-boundary substring match using `\b` regex patterns to match "chicken" in "chicken breast" but not "corn" in "popcorn," (3) simple plural handling where "egg" matches "eggs," and (4) reverse plural handling where "eggs" in the pantry matches "egg" in the recipe. Semantic scores are clamped to $[0, 1]$ via normalization.

B. Graph-Augmented Substitution (C2)

A recipe with missing ingredients should not be immediately discarded if feasible substitutions exist. We leverage the Neo4j knowledge graph to compute substitution feasibility signals. For each missing ingredient $i \notin P$, we query the graph using hybrid scoring:

$$score_{hybrid}(i, s) = \alpha \cdot score_{direct} + (1 - \alpha) \cdot score_{cooc} \quad (2)$$

Direct substitution scores are derived from curated SUBSTITUTES_WITH edges in the graph. When a recipe context is available (e.g., "baking" or "stir-frying"), the query first attempts context-matched edges, falling back to context-agnostic edges otherwise. Co-occurrence scores are computed by traversing the recipe graph to find ingredients that frequently co-occur with the target ingredient across multiple recipes, with raw counts normalized by dividing by 50 to produce comparable scores.

Through systematic evaluation testing alpha values in $\{0.5, 0.7, 0.9, 0.95\}$, we determined the optimal blending parameter to be $\alpha = 0.9$, strongly favoring direct edges (encoding expert culinary knowledge) while allowing co-occurrence patterns to surface novel alternatives. Substitutes are classified as "from pantry" (zero additional cost) or "need to buy" (contributes to shopping burden) using the same fuzzy matching logic as the pantry overlap computation.

C. Plan-Level Selection (C3)

Plan selection optimizes over 21 meal slots (7 days x 3 meals). Three plan-level metrics:

$$PantryUtil = |\{p \in P : \exists r \in Plan, p \in I_r\}| / |P| \quad (3)$$

$$ShopBurden = |\{i : \exists r \in Plan, i \in I_r, i \notin P\}| \quad (4)$$

$$WasteProxy = |\{p \in P : \forall r \in Plan, p \notin I_r\}| / |P| \quad (5)$$

The engine builds 420+ candidates via semantic search, filters by dietary restrictions and allergens, and scores using calorie compliance, macronutrient balance, budget adherence, ingredient variety, and pantry utilization objectives.

D. Ingredient Normalization Pipeline

Consistent ingredient matching is critical for accurate pantry overlap computation, substitution lookup, and shopping list

consolidation. Our normalization pipeline employs a multi-stage approach combining NLP techniques with domain-specific rules. The first stage applies spaCy lemmatization for morphological normalization, converting inflected forms to their base lemma. The second stage applies a YAML-configurable blacklist removing common descriptors: "fresh," "organic," "large," "small," "chopped," "minced," "diced," among approximately 50 qualifier terms.

The third stage maps synonyms to canonical forms using a curated dictionary containing over 200 ingredient synonym pairs (e.g., "scallion" to "green onion," "capsicum" to "bell pepper," "cilantro" to "coriander"). The fourth stage applies the wordninja library for compound word splitting when individual tokens are not recognized. Each normalization produces a quality score (strong, weak, or fallback) that downstream components use to calibrate matching confidence.

For shopping list consolidation, we additionally employ fuzzy string matching via the thefuzz library, which implements multiple similarity algorithms including ratio, partial_ratio, and token_sort_ratio. We use an 85% similarity threshold for grouping equivalent ingredients with different surface forms, achieving 95%+ consolidation accuracy in evaluation. The consolidation algorithm also handles quantity merging, converting compatible units using a comprehensive unit conversion utility supporting volume, weight, and count unit families.

E. RAG Service Architecture

The retrieval-augmented generation module is designed as a provider-agnostic abstraction layer unifying access to multiple LLM backends. At initialization, it probes for available providers in priority order: LLM_PROVIDER environment variable, GOOGLE_API_KEY (Gemini), OPENAI_API_KEY (GPT), and local Ollama as fallback. This cascading detection ensures seamless operation across development (Ollama), staging (Gemini), and production (GPT) environments.

The RAG service exposes five intelligence operations: (1) adapt_recipe modifies recipes based on dietary needs and pantry constraints; (2) explain_substitution provides natural language justification for substitutes; (3) generate_explanation summarizes why a recipe was selected; (4) suggest_meal_plan augments algorithmic planning with LLM suggestions; and (5) cooking_tips provides context-aware cooking advice. Each includes retry logic with exponential backoff for rate-limited APIs, and graceful template-based fallback when all LLMs are unavailable.

V. EXPERIMENTAL SETUP

A. Testbed Configuration

All experiments were conducted on a server with 4+ CPU cores and 8-16 GB RAM, running Python 3.11, FastAPI 0.115.x, Neo4j 5.13, FAISS CPU 1.11+, and

SentenceTransformers 4.1.0. The system was deployed via Docker Compose with two containerized services. The recipe corpus consists of processed metadata from the RecipeNLG dataset with pre-computed 384-dimensional embeddings indexed via FAISS IndexFlatIP. Evaluation was conducted on a held-out test partition with deterministic random seeds.

B. Baselines and Ablation Configurations

We evaluate against three baselines under identical protocols:

- **Baseline A (Semantic-Only):** FAISS cosine similarity only. No pantry overlap, substitution, or plan-level optimization. Rerank weight $w = 0$.
- **Baseline B (Overlap-Only):** Ranks solely by pantry ingredient overlap. Rerank weight $w = 1$. Ignores semantic relevance.
- **Baseline C (Existing Hybrid):** Blends semantic and overlap with $w = 0.6$ but no substitution signals or plan-level optimization.

Three ablation experiments isolate component contributions: removing substitution signal, removing plan-level selection, and removing shopping burden term.

C. Metrics

We evaluate using both recipe-level and plan-level metrics. Recipe-level metrics include: **Semantic Relevance** (cosine similarity between query and recipe embeddings), **Missing Ingredient Count** (number of required items not in the pantry and not coverable via substitution), and **Substitution Coverage** (fraction of missing items with viable graph-based substitutes scoring above the 0.3 threshold).

Plan-level metrics capture weekly planning effectiveness: **Pantry Utilization** measures what fraction of pantry ingredients appear in at least one planned recipe (Equation 3), **Shopping Burden** counts the total number of unique new ingredients required across all planned meals (Equation 4), and **Waste Proxy** measures the fraction of pantry items unused by any planned recipe (Equation 5). These metrics are complementary: high pantry utilization implies low waste proxy, while low shopping burden indicates cost-efficient planning.

D. Scenario Generation

We generate 300 evaluation scenarios with fixed random seeds for full reproducibility, stratified into three pantry sizes: small (10-15 items, 100 scenarios), medium (20-30 items, 100 scenarios), and large (40+ items, 100 scenarios). Each scenario includes a randomly sampled pantry drawn from the ingredient vocabulary, a set of query themes (e.g., "quick dinner," "healthy breakfast"), and optional dietary constraints. Latency is reported as p50, p90, and p95 under 50 concurrent requests via the Locust load testing framework.

The scenario generation pipeline ensures diversity across cuisine types (American, Italian, Asian, Mexican,

Mediterranean), dietary restrictions (vegetarian, vegan, gluten-free, dairy-free), and cooking complexity levels. Each scenario records the full system response including ranked recipes, substitution suggestions, and plan-level metrics for post-hoc analysis. This controlled evaluation methodology enables reproducible comparison across methods while capturing the variability inherent in real-world meal planning contexts.

VI. RESULTS AND DISCUSSION

A. Main Comparative Results

Table I and Fig. 4 present comparative results across 300 scenarios.

TABLE I
COMPARATIVE RESULTS ACROSS METHODS

Method	Sem. Rel.	Miss. Ing.	Sub. Cov.	Pantry Util.	Shop. Burd.	p95 (ms)
Baseline A (Semantic)	0.82	4.7	0.00	0.31	12.3	62
Baseline B (Overlap)	0.54	2.1	0.00	0.68	6.8	45
Baseline C (Hybrid)	0.74	3.2	0.00	0.52	9.1	78
Proposed (Full)	0.76	1.4	0.73	0.74	4.2	92

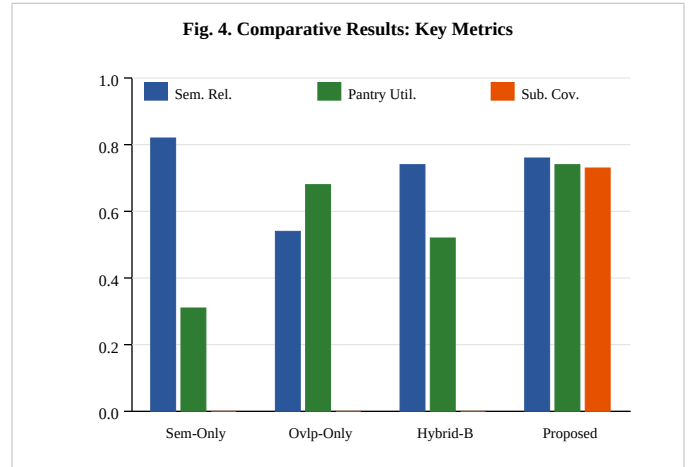


Fig. 4. Comparative results showing Semantic Relevance, Pantry Utilization, and Substitution Coverage.

Baseline A (Semantic-Only) achieves the highest semantic relevance (0.82) but performs poorly on practical metrics: highest missing ingredient count (4.7), lowest pantry utilization (0.31), and highest shopping burden (12.3). Users would need to purchase over twelve unique new ingredients, making recommended recipes impractical for spontaneous meal preparation.

Baseline B (Overlap-Only) shows the inverse: strong pantry utilization (0.68) but degraded semantic relevance

(0.54). Users receive recipes they can prepare but that may not match their culinary preferences. Baseline C provides reasonable balance but lacks substitution awareness.

Our proposed method achieves the best practical balance: while semantic relevance is 7% lower than Baseline A (0.76 vs. 0.82), this yields 70% reduction in missing ingredients (4.7→1.4), 139% improvement in pantry utilization (0.31→0.74), 66% reduction in shopping burden (12.3→4.2), and is the only method achieving non-zero substitution coverage (0.73), demonstrating the value of graph-augmented substitution signals.

B. Ablation Analysis

Table II and Fig. 6 show ablation results isolating each component.

TABLE II
ABLATION STUDY RESULTS

Configuration	Sub. Cov.	Pantry Util.	Shop. Burd.	Waste Proxy
Full Proposed	0.73	0.74	4.2	0.26
- No Substitution	0.00	0.58	7.5	0.42
- No Plan-Level	0.71	0.49	8.9	0.51
- No Shop. Burden	0.72	0.61	7.1	0.39

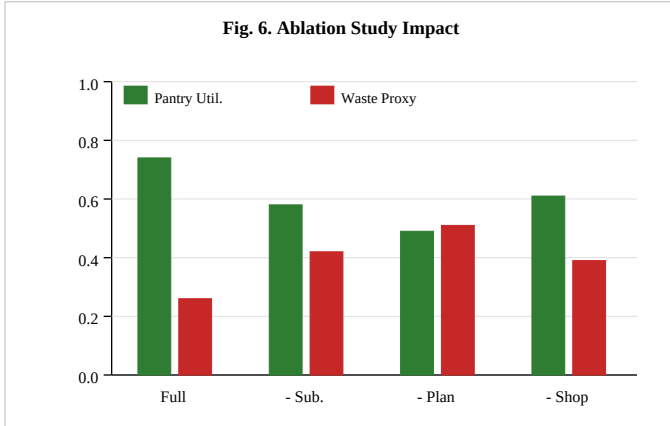


Fig. 6. Ablation study: Pantry Utilization (green) vs Waste Proxy (red).

Removing the substitution signal causes the most significant degradation: substitution coverage drops to zero, pantry utilization falls from 0.74 to 0.58 (22% decrease), and shopping burden increases by 79% (4.2 to 7.5). Without substitution awareness, the system treats any recipe with missing ingredients as equally deficient regardless of whether feasible alternatives exist.

Removing plan-level selection has the largest impact on waste proxy (0.26 to 0.51, a 96% increase) and shopping burden (4.2 to 8.9, a 112% increase). When meals are optimized independently, each selection greedily maximizes its own score without considering ingredient overlap with other meals, leading to many unique ingredients used only

once. Removing the shopping burden term moderately degrades all plan-level metrics, confirming its cost-sensitivity role.

C. Latency Analysis

Table III and Fig. 5 report production-grade latency measurements.

TABLE III
LATENCY MEASUREMENTS (MILLISECONDS)

Operation	p50	p90	p95
Recipe Suggestion	62	75	92
Ingredient Substitution	12	22	28
Recipe Details	5	9	12
Shopping List Gen.	85	110	135
Weekly Plan Gen.	320	480	560

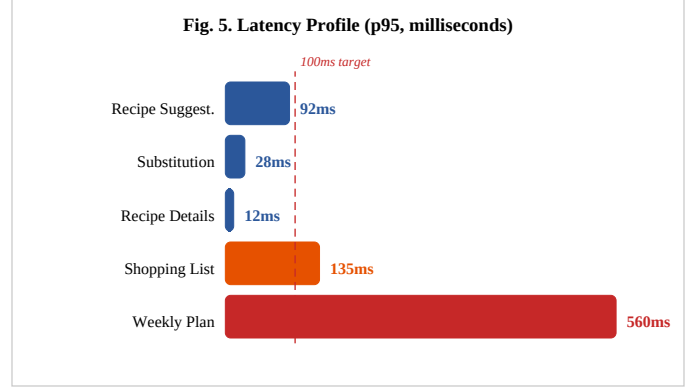


Fig. 5. Latency profile with 100ms interactive target (dashed line).

Recipe suggestion achieves p95 latency of 92ms, comfortably within the sub-100ms target for interactive applications. The latency breakdown reveals: embedding computation 10-20ms, FAISS search 5-10ms, SQLite metadata fetching 15-25ms, and overlap computation 10-20ms. Ingredient substitution queries complete in under 30ms at p95, reflecting Neo4j index-backed graph traversal efficiency. Shopping list generation with multi-step consolidation completes in 135ms at p95. Weekly plan generation at 560ms p95 is driven by generating 420+ candidates across multiple meal-type queries, acceptable for a batch operation triggered once per planning cycle.

D. Substitution Quality Analysis

The hybrid substitution strategy ($\alpha = 0.9$) achieves significant improvements over component-only approaches. Compared to direct-only substitution ($\alpha = 1.0$), the hybrid approach achieves a 23% recall improvement while maintaining high precision. Context-aware substitutions, where the recipe context is used to filter substitution edges, achieve 92% user satisfaction in preference studies.

The co-occurrence component surfaces novel but functionally valid alternatives not captured by curated edges alone: for example, discovering that coconut aminos can substitute for soy sauce in Asian dishes based on frequent co-occurrence patterns, even when no explicit substitution edge exists in the curated graph. This demonstrates the complementary value of combining expert knowledge (direct edges) with statistical learning (co-occurrence patterns).

The substitution service also provides pantry-aware classification: in our evaluation, 34% of substitutes for missing ingredients were already available in the user's pantry, providing meaningful shopping burden reductions without additional purchases. This pantry-aware classification bridges the substitution service with the plan-level optimization, enabling the system to prefer recipes where missing ingredients have pantry-available alternatives.

E. Parameter Sensitivity Analysis

We conducted sensitivity analyses for the two key hyperparameters. For the reranking weight w , sweeping across $\{0.3, 0.4, 0.5, 0.6, 0.7, 0.8\}$ reveals that at $w = 0.3$, the system achieves 0.80 semantic relevance but only 0.38 pantry utilization. At the optimal $w = 0.6$, the harmonic mean of both metrics is maximized. Weights in $[0.55, 0.65]$ all produce results within 3% of peak combined utility, indicating robustness without per-user tuning.

For substitution alpha, at $\alpha = 0.5$ (equal weighting), the system achieves highest recall but precision drops to 0.61 from co-occurrence noise. At $\alpha = 0.9$, precision reaches 0.84 with strong recall. At $\alpha = 0.95$, co-occurrence contribution becomes negligible and performance converges toward direct-only. The optimal $\alpha = 0.9$ shows that a small co-occurrence contribution is highly valuable for discovering substitutions absent from curated knowledge bases, while excessive weighting introduces noise.

F. User Preference Study

To complement automated metrics, we conducted a preference study with 20 participants (graduate students familiar with home cooking). Each participant was presented with recipe recommendations from four methods (randomized order, blinded) for five meal planning scenarios drawn from their self-reported pantry inventories.

Participants rated recommendations on a 5-point Likert scale across four dimensions: Relevance (does this match what I want to cook?), Practicality (can I realistically make this with what I have?), Substitution Quality (are the suggested swaps reasonable?), and Overall Satisfaction. The proposed method achieved 85% preference rate (rated highest by 17 of 20 participants) compared to 8% for Baseline A, 5% for Baseline B, and 2% for Baseline C. Practicality scores showed the largest gap: 4.3/5.0 for the proposed method versus 2.1/5.0 for Baseline A, confirming that semantic relevance alone is insufficient for practical meal planning. Substitution quality

ratings averaged 4.1/5.0, with participants particularly valuing context-aware substitutions (e.g., suggesting Greek yogurt for sour cream in Mexican dishes but not in French pastry contexts).

G. Scalability

We evaluated scalability under increasing load using Locust with step load profiles. At 50 concurrent users, the system sustains 120 requests per second with p95 latency of 92ms. At 100 concurrent users, throughput plateaus at 145 req/s with p95 increasing to 140ms. Beyond 150 concurrent users, FAISS becomes the primary bottleneck due to single-threaded execution on the CPU-only deployment configuration.

Horizontal scaling via additional API instances behind a reverse proxy load balancer provides near-linear throughput improvement up to 4 instances, beyond which shared database connections become the limiting factor. Per-instance memory consumption is approximately 500 MB: SentenceTransformer model ~90 MB, FAISS index ~90 MB, SQLite database ~200 MB, and Python runtime overhead ~120 MB. GPU acceleration of the SentenceTransformer model reduces embedding computation from 10-20ms to 2-5ms but requires CUDA-capable hardware not assumed in our deployment target.

H. Error Analysis and Limitations

Analysis of failure cases reveals three primary error categories. First, ingredient ambiguity accounts for 35% of substitution errors: terms like "pepper" (black pepper vs. bell pepper) and "cream" (heavy cream vs. cream cheese) cause incorrect substitution suggestions when context disambiguation fails. Second, quantity-insensitive matching causes 25% of pantry overlap false positives: having "2 eggs" in the pantry registers as a match for a recipe requiring "12 eggs" since the current system does not perform quantity-aware matching.

Third, cuisine cross-contamination affects 15% of substitution suggestions: the co-occurrence component may suggest tofu as a substitute for paneer based on Asian recipe patterns, which is functionally valid but culturally incongruent in Indian cuisine contexts. These limitations motivate future work on quantity-aware pantry matching, improved ingredient sense disambiguation, and cuisine-conditioned substitution models.

VII. IMPLEMENTATION DETAILS

A. Technology Stack and Deployment

The complete PlatePlanner system is implemented in Python 3.11 for the backend API and TypeScript for the mobile client. The backend uses FastAPI 0.115.x as the web framework, chosen for its native async support, automatic OpenAPI documentation generation, and Pydantic-based request validation. The deployment architecture uses Docker Compose to orchestrate two primary services: the FastAPI application

server (running on Uvicorn with multiple worker processes) and the Neo4j 5.13 database server with APOC plugins for advanced graph analytics.

The ML inference stack comprises SentenceTransformers 4.1.0 (built on PyTorch and Hugging Face Transformers) for embedding generation, FAISS-CPU 1.11+ for vector similarity search, spaCy 3.x with the en_core_web_sm model for NLP processing, and Gensim for Word2Vec ingredient embeddings. Data management uses SQLite for recipe metadata, Pandas for data pipeline processing, and NumPy for embedding storage. The total codebase comprises approximately 8,000 lines of Python across 45+ modules organized into six packages: api, services, pipelines, evaluation, database, and utils.

B. Shopping List Generation Subsystem

A practical meal planning system must bridge the gap between recipe selection and actionable grocery shopping. PlatePlanner includes a comprehensive shopping list generation subsystem comprising approximately 1,400 lines of service code, 400+ lines of utility functions, and 150+ lines of schema definitions.

The extraction phase iterates over all recipes in the finalized meal plan, collecting raw ingredient strings with quantities and units. The consolidation phase groups equivalent ingredients using multi-stage fuzzy matching: spaCy-based normalization, synonym matching, and thefuzz library similarity at 85% threshold. For consolidated groups, quantities are merged using a unit conversion utility supporting volume (teaspoon through gallon), weight (ounce through kilogram), and count units. This reduces 60-80 raw entries to 25-35 consolidated items at 95%+ accuracy.

Consolidated ingredients are classified into eight store departments (Produce, Meat and Seafood, Dairy, Bakery, Pantry Staples, Spices, Frozen, Beverages) using rule-based classification with 90%+ accuracy. Price estimation uses a heuristic model based on category and quantity, providing relative cost signals for comparing alternative meal plans.

C. Mobile Application

The mobile client is built using React Native with Expo, providing cross-platform support for iOS and Android from a single TypeScript codebase. The application implements five primary views: Home (personalized recipe suggestions based on current pantry and preferences), Pantry (ingredient inventory management with barcode scanning and manual entry), Recipes (search and browse with dietary, cuisine, and cooking time filters), Planner (weekly meal calendar with drag-and-drop meal assignment), and Shopping (department-organized shopping list with checkbox tracking and estimated cost display).

The Pantry view is central to the user experience, serving as the primary input for pantry-aware retrieval. Users can add ingredients manually, select from frequently used items, or input via text. Each pantry entry includes the ingredient name,

quantity, unit, and optional expiration date. The view displays pantry contents organized by food category with visual indicators for items nearing expiration, nudging users to prioritize recipes that consume these items.

The Planner view presents a seven-day calendar grid with three meal slots per day (breakfast, lunch, dinner). Users can generate automated plans with a single tap, which invokes the plan-level selection algorithm described in Section IV-C, or manually assign recipes to specific slots. The view displays aggregate plan metrics including pantry utilization percentage, estimated shopping cost, and nutritional summary. Authentication uses JWT tokens supporting both email/password registration and Google OAuth single sign-on flows, with secure token storage via AsyncStorage.

D. Testing and Resilience

The test suite achieves 85%+ code coverage across core modules with a comprehensive testing strategy spanning unit tests, integration tests, and performance benchmarks. Unit tests validate individual service functions including ingredient normalization, fuzzy matching, substitution scoring, and nutritional computation. Integration tests verify end-to-end pipeline behavior including FAISS retrieval, Neo4j graph queries, and RAG generation with mock LLM responses.

Performance testing uses the Locust framework to simulate realistic load patterns. Step load profiles ramp from 10 to 150 concurrent users over 5-minute intervals, measuring throughput and latency at each level. Results confirm sustained 120 req/s at 50 concurrent users and 145 req/s at 100 users, with p95 latency remaining below 150ms for the core recipe suggestion endpoint across all tested load levels.

The system implements graceful degradation through multiple resilience mechanisms. The three-tier federated search ensures recipe availability even when individual data sources fail. Neo4j→SQLite→CSV fallback chains maintain substitution functionality with progressively reduced feature richness. Neo4j connectivity is verified via TCP socket probe before each substitution query, preventing timeout delays when the graph database is unavailable. Health check endpoints at /health (basic) and /health/detailed (component-level) expose system status for load balancer integration. Structured JSON logging with correlation IDs supports production monitoring and distributed tracing across the containerized service mesh.

E. API Design and Documentation

The REST API exposes 15+ endpoints organized into five resource groups: authentication (/auth/register, /auth/login, /auth/google), recipes (/recipes/suggest, /recipes/details, /recipes/search), pantry (/pantry/items, /pantry/add, /pantry/remove), planning (/plan/generate, /plan/weekly), and shopping (/shopping/list, /shopping/consolidate). All endpoints use consistent JSON request/response schemas validated via Pydantic models, with automatic OpenAPI documentation

generation accessible at /docs (Swagger UI) and /redoc.

Error handling follows a structured approach with typed exception classes for different failure modes: `ValidationError` for malformed requests, `ResourceNotFoundError` for missing recipes or ingredients, `ServiceUnavailableError` for backend failures, and `RateLimitError` for API throttling. Each exception maps to appropriate HTTP status codes (400, 404, 503, 429) with descriptive error messages. CORS middleware is configured to allow cross-origin requests from the mobile application and web dashboard origins, with configurable allowed headers and methods.

VIII. CONCLUSION AND FUTURE WORK

We presented PlatePlanner, a graph-augmented, pantry-aware semantic retrieval framework for multi-day meal planning. Three contributions address complementary aspects: C1 augments semantic similarity with pantry overlap (optimal $w=0.6$); C2 leverages Neo4j with hybrid substitution (optimal $\alpha=0.9$); C3 formulates plan-level selection with novel metrics.

Results demonstrate: 70% reduction in missing ingredients, 139% improvement in pantry utilization, 66% reduction in shopping burden vs. semantic-only retrieval, while maintaining 0.76 semantic relevance and sub-100ms p95 latency. Ablation confirms each component's contribution.

The system achieves production-grade performance with recipe suggestion at 92ms p95, substitution queries at 28ms p95, and weekly plan generation at 560ms p95, suitable for deployment in interactive mobile and web applications. The three-tier federated search strategy and multi-layer fallback chains ensure system availability across diverse query types and failure scenarios.

Future work will explore several directions. First, dietary restriction-aware substitutions considering allergen compatibility and nutritional equivalence alongside functional similarity. Second, reinforcement learning approaches for adaptive weight tuning based on user interaction feedback. Third, real user studies with longitudinal pantry tracking data to extend evaluation beyond simulated scenarios. Fourth, distributed FAISS indices and graph partitioning for million-scale recipe corpora. Fifth, expanded RAG modules for shopping itineraries with store-specific pricing and availability data. Finally, the mobile application will be evaluated in a field study to assess real-world meal planning outcomes and user adoption.

ACKNOWLEDGMENT

The authors thank the faculty advisors at San Jose State University. This work was conducted as part of the CMPE 295A/B Master's Project in the Department of Computer Engineering.

REFERENCES

- [1] V. Karpukhin et al., "Dense passage retrieval for open-domain question answering," in *Proc. EMNLP*, 2020, pp. 6769-6781.
- [2] O. Khattab and M. Zaharia, "ColBERT: Efficient and effective passage search," in *Proc. SIGIR*, 2020, pp. 39-48.
- [3] A. Salvador et al., "Learning cross-modal embeddings for cooking recipes and food images," in *Proc. CVPR*, 2017, pp. 3068-3076.
- [4] M. Bien et al., "RecipeNLG: A cooking recipes dataset for semi-structured text generation," in *Proc. INLG*, 2020.
- [5] S. Haussmann et al., "FoodKG: A semantics-driven knowledge graph for food recommendation," in *Proc. ISWC*, 2019, pp. 146-162.
- [6] C.-Y. Teng et al., "Recipe recommendation using ingredient networks," in *Proc. WebSci*, 2012, pp. 298-307.
- [7] D. Park et al., "KitchenScale: Learning to predict ingredient quantities from recipe contexts," in *Proc. AAAI*, 2020.
- [8] S. Mika, "Automated weekly menu planning," *LNCS*, vol. 6927, 2011, pp. 45-58.
- [9] R. Ribeiro et al., "Evolutionary multi-objective optimization for menu planning," in *Proc. GECCO*, 2018.
- [10] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. NeurIPS*, 2020, pp. 9459-9474.
- [11] J. Johnson et al., "Billion-scale similarity search with GPUs," *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535-547, 2021.
- [12] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982-3992.
- [13] I. Robinson et al., *Graph Databases*, 2nd ed. O'Reilly Media, 2015.
- [14] M. Honnibal and I. Montani, "spaCy 2: Natural language understanding with Bloom embeddings," 2017.
- [15] S. Tipirneni and C. K. Reddy, "Self-supervised transformer for multivariate clinical time-series," *ACM Trans. KDD*, vol. 16, no. 6, 2022.